BTP - Caso de Uso: Tienda

Alberto Piedra Venegas

Octubre 2026

Índice

1. Servicio CAP **2**
- 1.1. Introducción y caso de uso . . . 2
- 1.2. Requisitos de la aplicación . . . 2
- 1.3. Preparar el entorno de desarrollo . . . 2
- 1.4. Crear el proyecto CAP desde cero . . . 3
- 1.5. Definir el modelo de datos . . . 4
- 1.6. Preparar y persistir los datos . . . 5
- 1.7. Publicar y probar OData V4 . . . 6

2. Interfaz con Fiori y uso de Draft **9**
- 2.1. Generar la aplicación Fiori Elements . . . 9
- 2.2. Mostrar las compras de la categoría . . . 10
- 2.3. Activar la gestión de borradores . . . 11
- 2.4. Pruebas funcionales . . . 12
  - 2.4.1. Creación de una categoría . . . 12
  - 2.4.2. Modificación de una categoría . . . 13
  - 2.4.3. Modificación de una compra desde la ficha . . . 13
  - 2.4.4. Validación del nombre obligatorio . . . 14
  - 2.4.5. Eliminación de una categoría . . . 15

3. Conclusiones **17**

1. Servicio CAP

1.1. Introducción y caso de uso

En este caso práctico construiremos una aplicación para una tienda de ropa y complementos utilizando SAP Cloud Application Programming Model (CAP), un servicio OData V4 y una aplicación SAP Fiori Elements. El recorrido que seguiremos es: modelo CDS, base de datos SQLite local, servicio OData V4, Fiori Elements y gestión de borradores (Draft).

La tienda necesita consultar sus categorías de producto y las compras realizadas por sus clientes, crear y modificar registros desde una interfaz web sencilla y gestionar las compras asociadas a cada categoría. Los datos proceden de un dataset de comportamiento de compra con 3 900 registros, del que tomaremos una muestra de 16 compras.

1.2. Requisitos de la aplicación

El modelo estará formado por dos entidades relacionadas:

Entidad	**Campos**	**Relación / regla**
Categorias	ID, nombre	El nombre es obligatorio. Una categoría contiene varias compras.
Compras	ID, articulo, importe, temporada, valoracion, ubicacion	Cada compra pertenece a una categoría (relación 1:N).

Cuadro 1: Entidades del modelo

- El proyecto expondrá un servicio llamado `TiendaService` y utilizará SQLite como base de datos local.
- Los datos de prueba serán 4 categorías y 16 compras.
- La interfaz tendrá una lista de categorías y una ficha de detalle con su tabla de compras.
- La edición utilizará Draft: crear, modificar, guardar y cancelar cambios.

1.3. Preparar el entorno de desarrollo

Utilizaremos SAP Business Application Studio (BAS) desde una cuenta de prueba (*trial*) de SAP BTP, con un Dev Space de tipo *Full Stack Cloud Application*. Antes de comenzar comprobaremos que la herramienta de línea de comandos de CAP está disponible mediante el comando `cds version`.

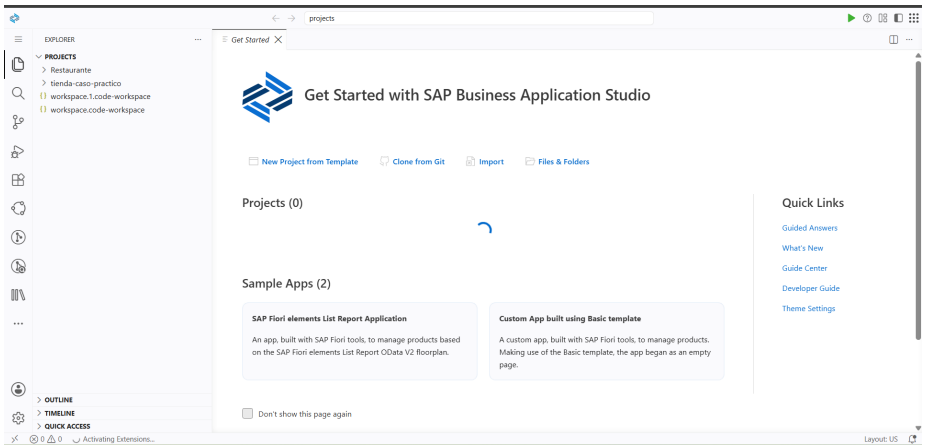


Figura 1: Comprobamos que el BAS se crea correctamente dentro de nuestro DEV Space

```
user: projects $ cds version

   @sap/cds-dk (global)  10.1.2   /extbin/globals/bun/global/install/node_modules/@sap/cds-dk
   cds.home                       /extbin/globals/bun/global/install/node_modules/@sap/cds
   cds.root                       ~/projects
   npm root -l                    ./node_modules
   npm root -g                    ~/.node_modules_global/lib/node_modules
   Node.js               24.21.0  /local/asdf-inst-inbox-runtime/installs/nodejs/24.21.0/bin/node
   npm                   11.19.0  ~/.asdf-inst/plugins/nodejs/shims/npm

user: projects $
```

Figura 2: BAS con el Dev Space en ejecución y salida de `cds version`

1.4. Crear el proyecto CAP desde cero

Para mantener la práctica separada de otros trabajos crearemos el proyecto `tienda-caso-practico`, que inicializaremos como proyecto CAP con el runtime de Node.js y SQLite:

```
mkdir tienda-caso-practico && cd tienda-caso-practico
cds init .
cds add nodejs
npm install
cds add sqlite
```

La estructura del proyecto contiene las siguientes carpetas principales:

- `db`: contiene el modelo de datos (CDS) y los datos de ejemplo (CSV).
- `srv`: contiene la definición del servicio OData y la lógica del backend.

- `app`: contiene la aplicación SAP Fiori Elements.
- `package.json`: gestiona la configuración y las dependencias del proyecto.

```
user: projects $ mkdir tienda-caso-practico && cd tienda-caso-practico
user: tienda-caso-practico $ cds init .

  Successfully initialized CAP project

user: tienda-caso-practico $ cds add nodejs

  Adding facet: nodejs

  Successfully added features to your project
```

Figura 3: Inicialización del proyecto con `cds init`

```
user: tienda-caso-practico $ ls
app  db  eslint.config.mjs  node_modules  package.json  package-lock.json  readme.md  srv
```

Figura 4: Estructura inicial del proyecto

1.5. Definir el modelo de datos

En el fichero `db/schema.cds` definiremos las entidades `Categorias` y `Compras`. La relación uno a muchos se modela con una *Composition* desde la categoría y una *Association* desde la compra. El nombre de la categoría será obligatorio mediante `not null`.

```
namespace tienda;

entity Categorias {
  key ID     : Integer;
  nombre     : String not null;
  compras    : Composition of many Compras
               on compras.categoria = $self;
}

entity Compras {
  key ID      : Integer;
  articulo    : String;
  importe     : Integer;
  temporada   : String;
  valoracion  : Decimal(2,1);
  ubicacion   : String;
  categoria   : Association to Categorias;
}
```

Una *Composition* expresa que la entidad hija pertenece a la padre: las compras existen dentro de una categoría y se gestionan junto con ella. La *Association* de la compra es la referencia inversa hacia su categoría.

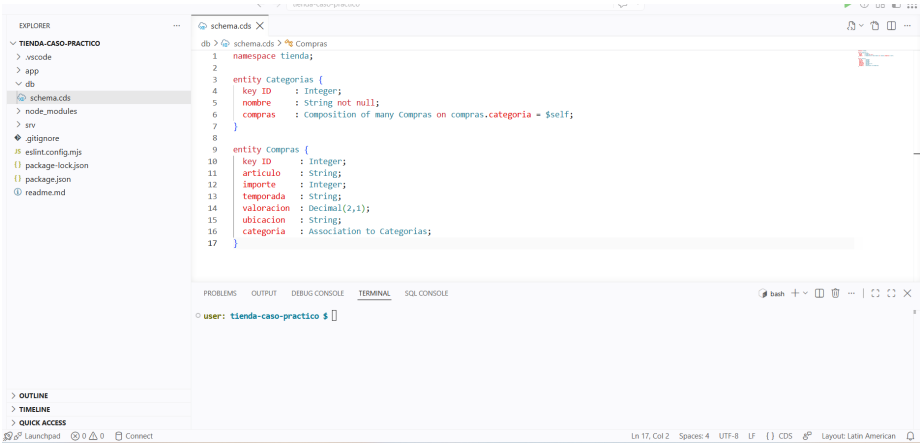


Figura 5: Modelo CDS con la Composition y la Association

1.6. Preparar y persistir los datos

Cargaremos los datos de ejemplo mediante dos ficheros CSV dentro de `db/data`, nombrados según el patrón `namespace-Entidad.csv`: `tienda-Categorias.csv` con las 4 categorías y `tienda-Compras.csv` con las 16 ejemplos de compras del dataset de partida.

```
ID,nombre
1,Clothing
2,Footwear
3,Outerwear
4,Accessories
```

```
ID,articulo,importe,temporada,valoracion,ubicacion,categoria_ID
1,Blouse,53,Winter,3.1,Kentucky,1
2,Sweater,64,Winter,3.1,Maine,1
3,Jeans,73,Spring,3.1,Massachusetts,1
...
```

La columna `categoria_ID` es la clave foránea que CAP genera a partir de la asociación `categoria`. Después configuraremos SQLite para que use un fichero persistente en el `package.json`:

```
"cds": {
  "requires": {
    "db": { "kind": "sqlite",
            "credentials": { "url": "db.sqlite" } }
  }
}
```

Y desplegaremos el modelo:

```
cds deploy
```

El mensaje `successfully deployed to db.sqlite` confirma que la base de datos se ha creado como fichero y que los datos de los CSV se han cargado. Los CSV solo se cargan durante el despliegue, por lo que tras cualquier cambio en el modelo o en los datos habrá que borrar `db.sqlite` y volver a ejecutar `cds deploy`.

Para comprobar que los datos se han cargado y persisten en el fichero, consultamos directamente `db.sqlite` desde la terminal con el cliente `sqlite3`. El recuento de la tabla `tienda_Compras` devuelve 16 registros, que coinciden con las compras de nuestro CSV de ejemplo.

```
sqlite3 -header -column db.sqlite "SELECT COUNT(*) FROM tienda_Compras;"
```

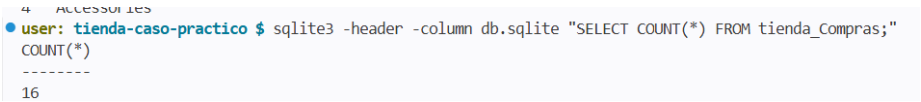


Figura 6: Consulta a `db.sqlite` con el número de compras cargadas

1.7. Publicar y probar OData V4

Definiremos el servicio en el fichero `srv/tienda.cds`:

```
using tienda from '../db/schema';

service TiendaService {
  entity Categorias as projection on tienda.Categorias;
  entity Compras    as projection on tienda.Compras;
}
```

Al llamar al servicio `TiendaService`, CAP lo publica en la ruta `/odata/v4/tienda`. Es importante que el nombre del servicio sea distinto del *namespace* del modelo (`tienda`): si coincidieran salvo por las mayúsculas, el despliegue fallaría porque SQL no distingue entre ambos nombres.

Con `cds watch` arrancaremos el servicio y consultaremos las entidades desde el navegador:

- `/odata/v4/tienda/Categorias`
- `/odata/v4/tienda/Compras?$top=5`
- `/odata/v4/tienda/Categorias?$expand=compras`

La última consulta devuelve cada categoría junto con sus compras, lo que confirma que la relación 1:N funciona correctamente.



Figura 7: Servicio arrancado con `cds watch` y respuesta de `Categorias`

```
{
    "@odata.context": "$metadata#Categorias",
    "value": [
        {
            "ID": 1,
            "nombre": "Clothing",
            "compras": [
                {
                    "ID": 1,
                    "articulo": "Blouse",
                    "importe": 53,
                    "temporada": "Winter",
                    "valoracion": "3.1",
                    "ubicacion": "Kentucky",
                    "categoria_ID": 1
                },
                {
                    "ID": 2,
                    "articulo": "Sweater",
                    "importe": 64,
                    "temporada": "Winter",
                    "valoracion": "3.1",
                    "ubicacion": "Maine",
                    "categoria_ID": 1
                },
                {
                    "ID": 3,
                    "articulo": "Jeans",
                    "importe": 73,
                    "temporada": "Spring",
                    "valoracion": "3.1",
                    "ubicacion": "Massachusetts",
                    "categoria_ID": 1
                },
                {
                    "ID": 5,
                    "articulo": "Blouse",
                    "importe": 49,
                    "temporada": "Spring",
                    "valoracion": "2.7",
                    "ubicacion": "Oregon",
                    "categoria_ID": 1
                }
            ]
        },
```


Figura 8: Consulta con `$expand=compras` mostrando la relación

2. Interfaz con Fiori y uso de Draft

2.1. Generar la aplicación Fiori Elements

Desde BAS generaremos una aplicación SAP Fiori Elements de tipo *List Report Page* conectada al servicio `TiendaService`, utilizando como origen de datos el proyecto CAP local. Como entidad principal elegiremos `Categorias`, con navegación a `compras`. La lista mostrará las cuatro categorías obtenidas del servicio OData V4.

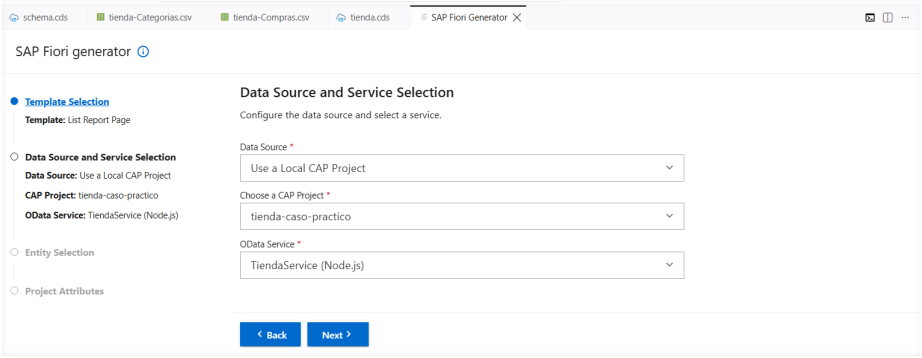


Figura 9: Elegimos el *Data Source* de donde sacará nuestros datos

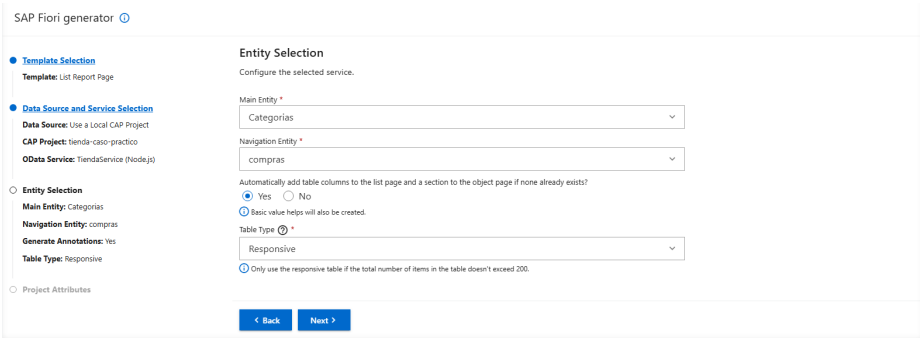


Figura 10: Cargamos la *entidad principal y de navegación*

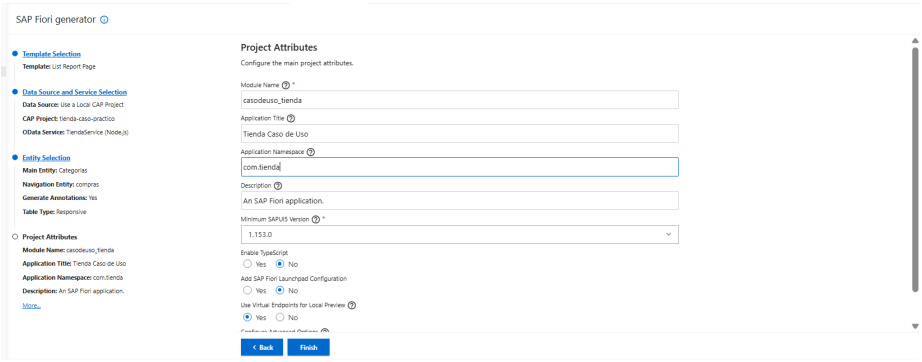


Figura 11: Añadimos *atributos a nuestro proyecto*

2.2. Mostrar las compras de la categoría

Mediante anotaciones UI en el fichero `annotations.cds` de la aplicación configuraremos la ficha (*Object Page*) de la categoría. Un *facet* de tipo referencia apunta a la lista de compras a través de la Composition, de modo que la ficha muestra los datos de la categoría y una tabla con sus compras:

```
UI.Facets : [
  { $Type  : 'UI.ReferenceFacet',
    ID     : 'GeneratedFacet1',
    Label  : 'Información general',
    Target : '@UI.FieldGroup#GeneratedGroup' },
  { $Type  : 'UI.ReferenceFacet',
    ID     : 'ComprasFacet',
    Label  : 'Compras',
    Target : 'compras/@UI.LineItem' }
]
```

La tabla de compras se define con otra anotación `UI.LineItem` sobre la entidad `Compras`, que establece las columnas: artículo, importe, temporada, valoración y ubicación. La ficha de la propia compra dispone además de su grupo de campos con el detalle completo.

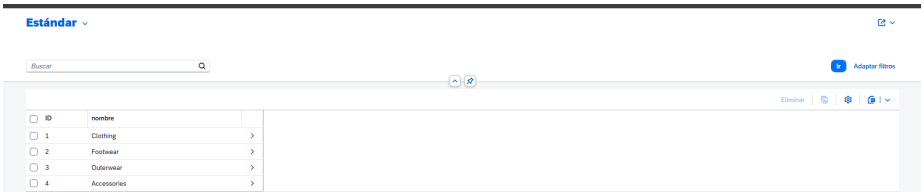


Figura 12: Se muestran las categorias

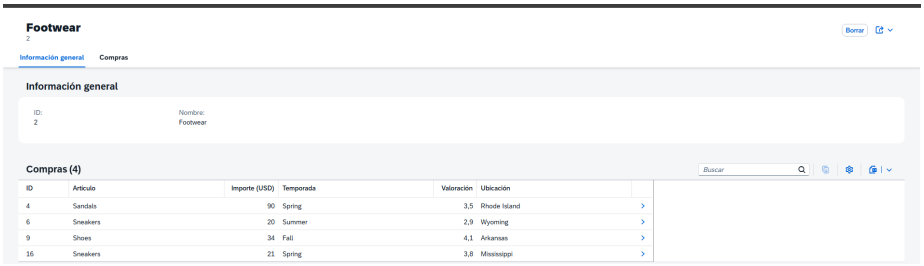


Figura 13: Ficha de una categoría con su tabla de compras

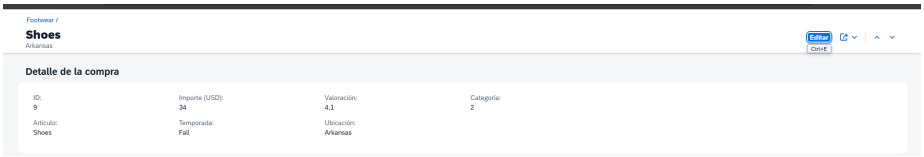


Figura 14: Ficha de detalle de una compra

2.3. Activar la gestión de borradores

Activaremos Draft en la entidad `Categorias` con la anotación `@odata.draft.enabled` y volveremos a ejecutar `cds deploy` para actualizar la estructura de SQLite:

```
service TiendaService {
  @odata.draft.enabled
  entity Categorias as projection on tienda.Categorias;
  entity Compras    as projection on tienda.Compras;
}
```

Draft mantiene una versión temporal del registro durante la edición. El usuario puede modificar los datos de la categoría y de sus compras y decidir después si los guarda o los descarta, sin tocar el registro activo hasta que confirma. CAP almacena esa versión temporal en tablas de borradores adicionales, por lo que, al cambiar el modelo o activar Draft, hay que regenerar la base de datos para que existan.

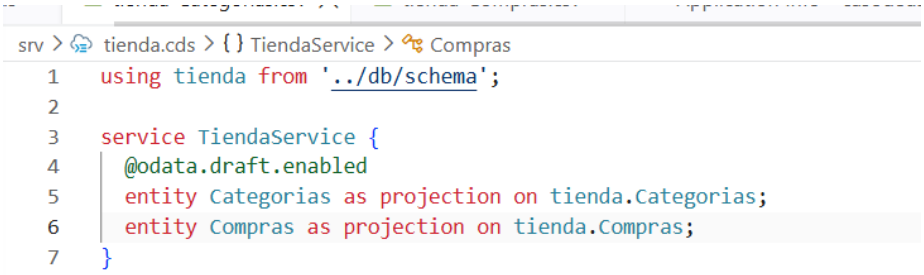


Figura 15: Anotación de Draft en el servicio

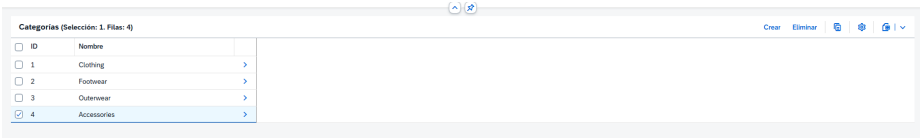


Figura 16: Botones de crear, editar, guardar y cancelar en la interfaz

2.4. Pruebas funcionales

Realizaremos las siguientes pruebas desde la interfaz.

2.4.1. Creación de una categoría

Crearemos una nueva categoría desde el botón *Crear* de la lista y la guardaremos.

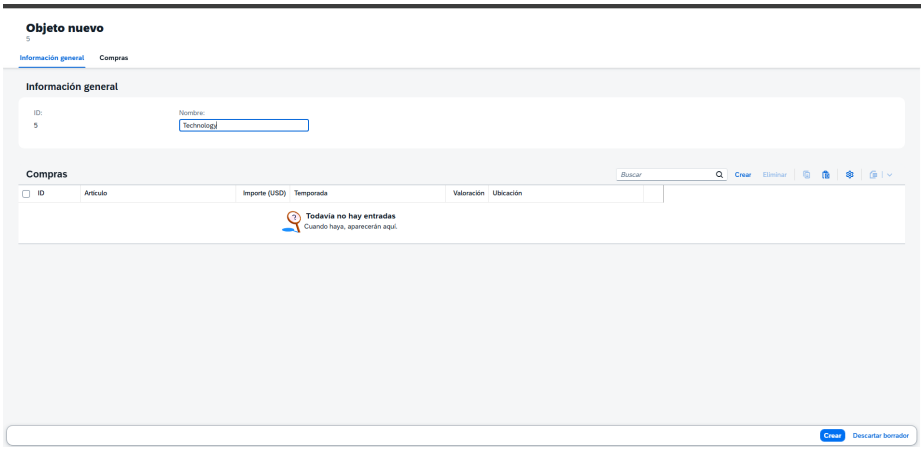


Figura 17: Añadimos una categoría nueva

2.4.2. Modificación de una categoría

Cambiaremos el nombre de la categoría `Footwear`. Tras guardar y volver a abrir el registro, los cambios se conservan.

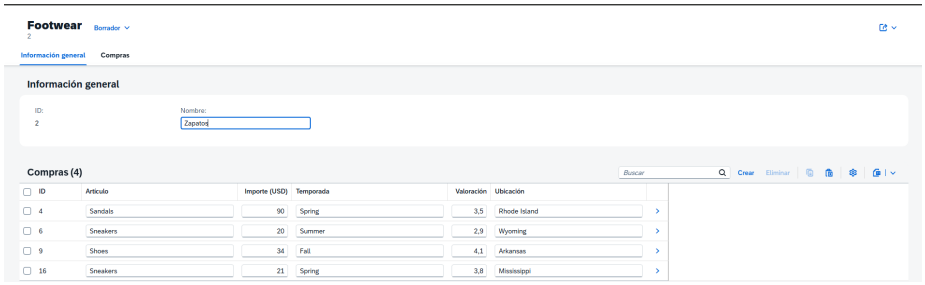


Figura 18: Modificamos el nombre de una categoría

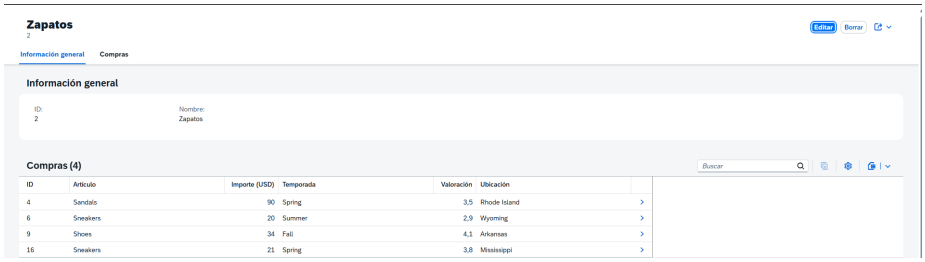


Figura 19: Comprobamos que se guardan los cambios

2.4.3. Modificación de una compra desde la ficha

Abrimos la categoría `Accessories` en modo edición. La pestaña *Compras* muestra su tabla de compras, que también es editable mientras trabajamos con el borrador.

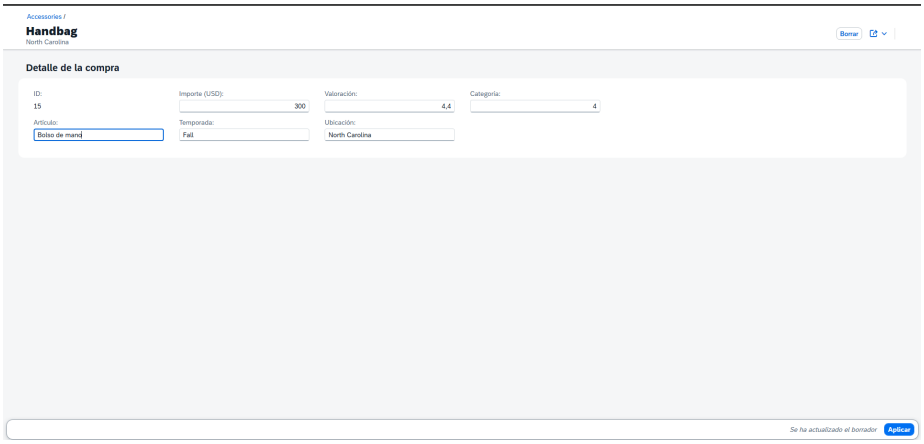


Figura 20: Tabla de compras de la categoría `Accessories` en modo edición

Entramos en el detalle de la compra con ID 15 y modificamos su artículo, que pasa de `Handbag` a `Bolso de mano`, y su importe, que pasa a 300 USD. Al pulsar *Aplicar*, el cambio se incorpora al borrador de la categoría; la aplicación indica que *se ha actualizado el borrador*. Los cambios solo se guardan definitivamente en la base de datos cuando guardamos la categoría.

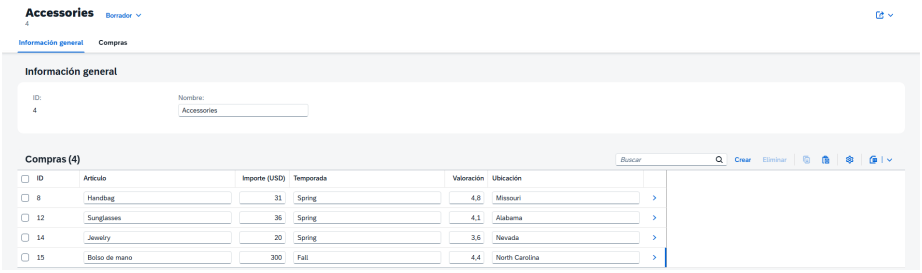


Figura 21: Comprobamos la modificación del nombre del accesorio

2.4.4. Validación del nombre obligatorio

Para comprobar la restricción obligatoria dejamos vacío el campo *Nombre* de la categoría `Accessories` dentro del borrador e intentamos guardar.

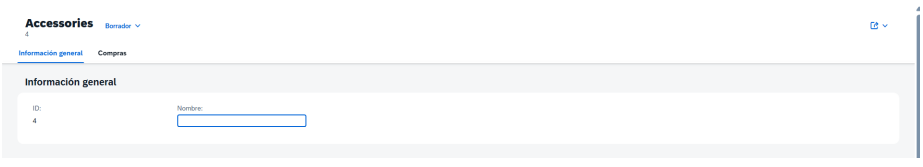


Figura 22: Categoría con el campo *Nombre* vacío

La aplicación rechaza el guardado y muestra el mensaje `NOT NULL constraint failed: tienda_Categorias.nombre`. Es la restricción `not null` del modelo, que se aplica en la propia base de datos, y confirma que no se puede guardar una categoría sin nombre. Como el registro sigue en estado de borrador, el valor original no se modifica.

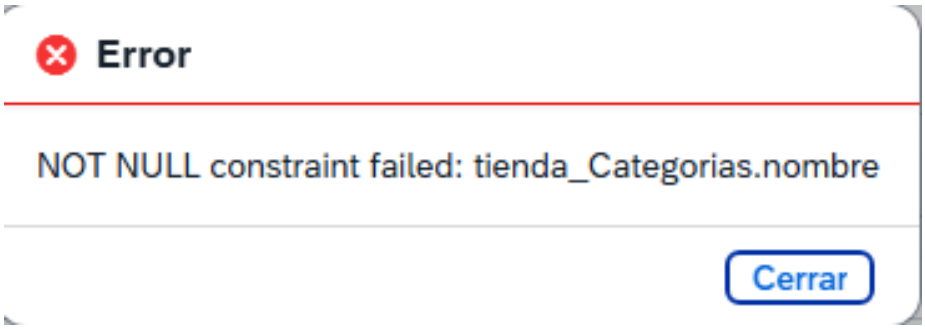


Figura 23: Error al guardar una categoría sin nombre

2.4.5. Eliminación de una categoría

Eliminaremos la categoría `Technology` que acabamos de crear y comprobaremos que se elimina correctamente.

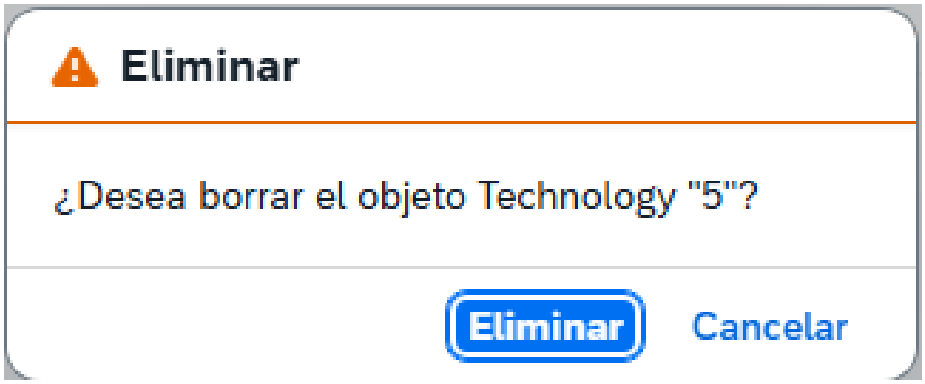


Figura 24: Eliminamos la categoría que hemos creado

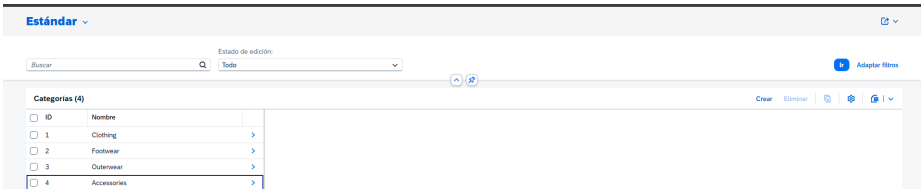


Figura 25: Comprobamos que se elimina correctamente la categoría

3. Conclusiones

¿Qué hace CAP? CAP permite definir el modelo de datos, las relaciones y los servicios mediante CDS. A partir de ese modelo se conecta con la base de datos y expone las entidades mediante OData V4, incluyendo las operaciones de creación, lectura, modificación y borrado (CRUD) con los métodos HTTP estándar, lo que reduce gran parte del código de backend.

¿Qué aporta Fiori Elements frente a crear la interfaz a mano? Fiori Elements genera una interfaz empresarial a partir del servicio OData y de sus anotaciones. En esta práctica nos ha permitido obtener la lista de categorías, la ficha de detalle, formularios y tablas sin programar manualmente cada control SAPUI5, y garantiza un aspecto y un comportamiento coherentes con el resto de aplicaciones SAP.

¿Para qué sirve Draft? Draft mantiene una versión temporal durante la edición. El usuario puede realizar cambios y decidir después si los guarda o los cancela, lo que resulta especialmente útil cuando un objeto contiene datos relacionados, como las compras de una categoría.

La práctica ha recorrido el flujo completo CAP local, SQLite, OData V4, Fiori Elements y Draft, y nos ha permitido comprobar cómo se integran las distintas capas de una aplicación empresarial.